How to Do This — A Stage-by-Stage Learning Guide to Building the Minorities Early-Warning System

From Raw Data to Live Dashboard: Every Technique, Every Choice, Every Trade-off Unpacked

Internship Project — Minorities Early-Warning System

June 2026

Contents

How to read this document

This is a **learning guide**, not a report. The other five deliverable documents (policy brief, detailed report, research design report, data dictionary, timeline) explain *what we found*. This document explains *how a reader could rebuild it from scratch* and *why we made every choice we made*.

We have written it in **twelve learning stages**. Each stage:

1. States the goal in one paragraph (what we are trying to do).
2. Explains the theory and any ML / statistics / NLP concept you need to follow it, broken down to the level where every parameter and every step is explained — not just "we used scikit-learn", but "this is what Logistic Regression is doing internally, step by step".
3. Lists the concrete tools, libraries and files we used.
4. Walks through what we did, step by step, with code-level snippets.
5. Explains **why we chose this approach and not other approaches we considered**.
6. Ends with a worked example so you can sanity-check your own reimplementation.

If you have never touched machine learning before, **read every "What this technique really does" box**. They never assume prior ML knowledge. We define every term the first time it appears.

If you already know ML, skim those boxes and focus on the "why-this-and-not-that" tables.

Stage 0 — The big picture before we touch any code

What we are building, in one diagram

```
Emergency-15 phone calls (Punjab-wide, 5.7M all-cause rows)
 |
 | filter: religious-offence tag OR minority keywords
 v
 minority_incidents (4,110 rows from Jan 2024 onward)
 |
 +---------+---------+
 | |
 v v
 per-case features per-(police-station x week) features
 (one row per call) (one row per PS per week, 656 PSes)
 | |
 v v
 per-case classifier PS-week forecaster
 (is this call (next 30 days: which PSes are at risk?)
 a minority attack?) |
 | v
 | outputs/.../psweek_forward.csv (656 rows)
 v
 outputs/.../predictions.csv (4,110 rows)
 |
 +-----+-----+
 |
 v
 Leaflet + Chart.js dashboard (HTML)
 +
 Streamlit app (CSV-driven)
```

What every box in this diagram is

The diagram has eight boxes. Here is what each one is, what produces it, and what consumes it.

Emergency-15 calls — the source

Punjab's `Emergency-15` helpline is the equivalent of dialling 911 in the US or 1122 in other Pakistani provinces. Every call gets logged in a relational database (Postgres) into one giant table called `<source_call_log_table>` in the `<source_db>` database. There are 5.7 million rows. Each row is one call, with columns including `call_id`, `level1` (broad category), `<category_lvl2>` (specific category), `description` (operator's free-text notes), `latitude`, `longitude`, `district`, `tehsil` (sub-district), `police_station`, `call_received_dt`, `response_dt`, and a handful of others.

filter — rule-based candidate selection

Most of those 5.7M calls are not about minorities at all (they are traffic incidents, domestic disputes, medical emergencies, etc.). We filter down to candidate minority-related rows using **three rules** joined by `OR`:

1. The `<category_lvl2>` tag is exactly `'Religious Offences'`, OR
2. The free-text `description` matches any minority-community word (Christian, Hindu, Sikh, Ahmadi, and their Urdu equivalents), OR
3. The free-text `description` matches a religious-place or religious-claim word (church, mandir, gurdwara, blasphemy, *toheen*).

This filtering is rule-based, not ML. There is no model involved. The output is a Python list of `call_id`s that pass the filter.

minority_incidents — the central clean table

Everything downstream reads from this table. 4,110 rows, 23 columns. About 928 of those rows are from Lahore. We also add one extra column called `is_minority_targeted`, which is `1` only when the `<category_lvl2>` tag is `'Religious Offences'` AND a minority keyword appears. This stricter column applies to **312** of the 4,110 rows and is the **target** the per-case classifier learns to predict.

per-case features — clues attached to each call

For every row in `minority_incidents`, we compute about 30 extra columns capturing different signals: how busy this police station has been lately, how close the call is to a religious holiday, how fast the local police usually respond, etc.

per-(PS, week) features — clues attached to each PS for each week

A different rectangular table where each row is one (police station, week) pair. About 656 PSes times roughly 120 weeks of training data gives about 82,000 rows. For each row we attach features that summarise the past, plus a label that captures the future: did a minority incident actually happen in this PS in the 30 days after this week?

per-case classifier — Logistic Regression + Random Forest

Two models that both answer the same question: *"given the features of this one phone call, what is the probability that it is a true minority-targeted incident?"* Each outputs a number between 0 and 1.

PS-week forecaster — Logistic Regression

One model that answers: *"for each (PS, week) row, what is the probability that a minority incident will occur in the 30 days after the start of this week?"* Once trained, we run it on a "today" row for every PS in Punjab and get a 656-row CSV ranked by risk.

Dashboard + Streamlit — the user-facing interface

A single self-contained HTML file (Leaflet + Chart.js, no server) shows the map, the forecast, and the per-case scores. A Streamlit web app does the same thing in a more interactive way for local use.

How an ML system is layered

Every real ML system, including this one, has the same five layers. Knowing them helps you orient when you read the code:

Layer	Question it answers	Where it lives in our project
Data layer	Where do the rows come from?	the incidents-table build script (full project ships CSV snapshots alongside)
Feature layer	What clues are attached to each row?	the project source code
Model layer	How do clues map to a prediction?	the project source code
Serving layer	How does the prediction reach the user?	the dashboard builder script, `app/`
Monitoring layer	Is the model still working in production?	model cards + system audit doc (full project)

If a future maintainer asks "where does feature X come from?", the answer is always in the feature layer. If they ask "why did the model say risk 0.94 for Mughalpura?", the answer involves both feature and model layers. If they ask "why does the dashboard look like this?", that is serving.

Where the code lives

```
minorities/<full project root>/
 data/
 csv_snapshot/ <- all 11 DB tables exported as CSV
 processed/ <- intermediate feature tables

 pipelines/ <- one file per pipeline step
 features/ <- one file per feature family
 models/ <- training + prediction scripts
 viz/ <- dashboard builder
 forecaster/ <- Prophet time-series model
 outputs/
 models/ <- trained .pkl model files
 reports/ <- model cards + metrics JSON
 dashboards/ <- the final HTML dashboard
 figures/ <- EDA plots
 deliverables/ <- the 6 reports including this guide
 app/ <- the Streamlit version of the dashboard
 docs/ <- phase-by-phase technical documentation
```

Stage 1 — Get the data (Data Engineering)

Goal of this stage

Pull the raw Emergency-15 calls out of the institute's Postgres database, filter them down to the religious-minority subset, parse every date carefully, and load the result into a clean, well-typed table called

`minority_incidents`.

This is the **most important step in the entire project.** Everything downstream — every feature, every model, every prediction, every dashboard widget — is computed off this one table. If we get this table wrong, nothing else can be right.

What "data engineering" really is

Before any ML, there is *data engineering*. It is the unglamorous work of:

- pulling rows out of one database,
- parsing dates someone stored as text,
- handling missing values,
- assigning a stable primary key to every row,
- and writing the cleaned-up version into a new table.

Roughly **70-80 percent of a data scientist's day** is spent on data engineering, not ML. A model is only as good as the data it is trained on. *Garbage in, garbage out.*

Concept primer — what is a relational database?

A relational database is a software system that stores **tables** (rectangles of rows and columns) and lets you query them with a language called **SQL**. The most common open-source one is **PostgreSQL** (often shortened to "Postgres"). Both `<source_db>` and `db_predictive_policing` in our project are Postgres databases.

Why use a database instead of CSV files?

- **Concurrent access.** Many users can read and write at the same time without trampling each other's edits.
- **Indexing.** A well-chosen index makes a query that would scan 5.7M rows in 30 seconds return in 30 milliseconds.
- **Types.** Each column has a fixed type (text, integer, timestamp, geometry) that the DB enforces. You cannot accidentally write the string `"hello"` into an integer column.
- **Transactions.** A group of operations either all succeed or all fail. You cannot leave the database half-updated.

CSV files have none of these. They are fine for small projects but fall over fast at scale.

Concept primer — what is SQL?

SQL stands for "Structured Query Language". It is the dialect we use to talk to a relational database. The four verbs you need for this project are:

- `SELECT` — read rows
- `INSERT` — write new rows
- `UPDATE` — modify existing rows
- `DELETE` — remove rows

Plus the filter clause:

- `WHERE` — restrict which rows you want

A toy example:

```
SELECT call_id, description
FROM response_time
WHERE <category_lvl2> = 'Religious Offences'
LIMIT 5;
```

This reads the first 5 rows from `<source_call_log_table>` whose `<category_lvl2>` tag is exactly the string `'Religious Offences'`, returning two columns.

Concept primer — connecting from Python

There are two libraries we use:

- **psycopg2** — the low-level driver that talks the Postgres wire protocol. You almost never use it directly.
- **SQLAlchemy** — a higher-level library that wraps `psycopg2` and gives you an "engine" object that pandas understands.

```
from sqlalchemy import create_engine

src = create_engine(
 "postgresql+psycopg2://USER:PASS@HOST:5432/<source_db>"
)
```

The big string above is a **connection URL**. Its parts:

- `postgresql+psycopg2://` — protocol + driver
- `USER:PASS` — credentials
- `HOST:5432` — server address and port (5432 is Postgres' default)
- `<source_db>` — database name

Once you have the engine, pandas can run any SQL through it:

```
df = pd.read_sql("SELECT * FROM response_time LIMIT 1000", src)
df` is now a pandas DataFrame with 1000 rows.

## Concept primer - what is a pandas DataFrame?

A **DataFrame** is pandas' name for a rectangular table held in memory.

- Rows are indexed (by default 0, 1, 2, ... but can be anything).
- Columns are named.
- Every column has a single dtype (`int64`, `float64`, `object` for
 strings, `datetime64`, `bool`, etc.).
- You can slice it, filter it, group it, join it, all in one or two
 lines of Python.

```python
df.shape # (rows, columns)
df.head(3) # first 3 rows
df["district"] # one column (a Series)
df[df["district"] == "lahore"] # filter
df["district"].value_counts() # group and count
df.groupby("district").size() # same idea
```
```

```
df.merge(other, on="call_id", how="left") # join
df.to_csv("out.csv", index=False) # write
```

If you have never used pandas: **a DataFrame is to Python what a sheet is to Excel**. Same mental model, faster and scriptable.

Filtering — what does the regex actually do?

This is the heart of Stage 1, and worth slowing down for.

We use a Postgres regex match to find minority-related descriptions. The full filter SQL is:

```
SELECT * FROM response_time
WHERE <category_lvl2> = 'Religious Offences'
 OR description ~* '\m(christian|isaai|masih|hindu|sikh|ahmadi|qadiani)\M'
 OR description ~* '\m(church|mandir|gurdwara|blasphemy|toheen)\M';
```

Let us walk through every character of the regex.

`~*` — this is the Postgres operator for "regex match, case insensitive". The `~` is the regex-match operator; the `*` is the modifier that makes it case insensitive. So `'HinDu' ~* 'hindu'` returns `true`.

`\m` — a Postgres-specific shorthand for "start of word". It only matches at the position immediately before the first letter of a word.

`(christian|isaai|masih|...)` — a **group** containing **alternatives**. The pipe `|` means "or". This matches if any of the listed words appears.

`\M` — Postgres-specific shorthand for "end of word". Matches at the position right after the last letter of a word.

Why word boundaries matter. Without `\m`/`\M`, the pattern `hindu` would match inside the word `Mahindra` (the Pakistani-Indian SUV brand). With word boundaries, it matches only the standalone word "hindu", "Hindu", "HINDU", etc.

Why we have Urdu/Punjabi transliterations. Callers and operators often write in *Roman Urdu* — Urdu words spelled with Latin letters. A caller might say "isaai" (the Urdu word for Christian) rather than "Christian". Without transliteration coverage we would silently drop those cases.

Why we used Postgres regex and not Python regex. Two reasons:

1. **Speed.** Filtering 5.7M rows in Postgres takes seconds; pulling them all into Python first and then filtering would take minutes and a lot of RAM.
2. **Indexing potential.** With a GIN trigram index on `description`, the filter can be made even faster. We did not need to go that far, but the option is there.

Building the strict label

Not every minority-keyword match is actually a minority-targeted incident. Sometimes the word appears in passing ("the caller's family is Christian but the complaint is a noise dispute"). So we built a **stricter** label that requires BOTH conditions:

```
import re
MINORITY_RE = re.compile(
 r"\b(christian|isaai|masih|hindu|sikh|ahmadi|qadiani)\b",
 flags=re.IGNORECASE,
)
```

```
df["is_minority_targeted"] = (
 (df["<category_lvl2>"] == "Religious Offences")
 & df["description"].fillna("").str.contains(MINORITY_RE, regex=True)
).astype(int)
```

Notice three things:

1. `\b` is Python's word boundary (works in any major regex flavour, unlike Postgres' `\m`/`\M`).
2. `re.IGNORECASE` flag — case insensitive match.
3. `.fillna("")` — defensive, because some descriptions are `NULL`. If we did not fill NaN, `.str.contains` would return `NaN` (not `False`), and our final `.astype(int)` would crash.

312 of the 4,110 rows satisfy this stricter rule. This is the column our per-case model will try to predict.

Parsing dates — the silent killer

The source DB stores dates as **text strings**, not as native timestamp columns. We saw three formats showing up in production:

- `2024-01-15 14:30:00`
- `01/15/2024 14:30`
- `15-01-2024 14:30:00`

Pandas has `pd.to_datetime(raw, errors='coerce')` which sounds convenient but is dangerous when you have multiple formats. It tries its best guess and falls back to `NaT` (pandas' name for "missing date") on failure. If most rows are in the *second* format and you have not told pandas to expect it, you can lose 90% of your rows silently.

Our solution:

```
from datetime import datetime

def parse_dt(raw):
 if raw is None or (isinstance(raw, float) and np.isnan(raw)):
 return None
 for fmt in (
 "%Y-%m-%d %H:%M:%S",
 "%m/%d/%Y %H:%M",
 "%d-%m-%Y %H:%M:%S",
 ):
 try:
 return datetime.strptime(str(raw).strip(), fmt)
 except ValueError:
 continue
 return None
```

The function tries each known format. If all three fail it returns `None` (which becomes `NaT` in the DataFrame), and **we then check** how many `NaT` we have and treat anything above 0 as a bug in the loader.

Format string codes for reference:

Code	Means	Example
`%Y`	4-digit year	`2025`
`%m`	2-digit month	`01`
`%d`	2-digit day	`15`
`%H`	24-hour	`14`
`%M`	minute	`30`
`%S`	second	`00`

Pitfall we hit in development. Our first loader used `pd.to_datetime(raw, errors='ignore')`. Pandas silently swallowed every unparseable row and left `incident_date` as `NaT`. After loading, **all 4,110 rows** had a NaT date. We only caught this when a downstream feature failed with "no valid dates in window". Lesson: **fail loudly, not silently**.

Schema enforcement on write

When we write the cleaned DataFrame into the working DB, we use SQLAlchemy's `to_sql`:

```
dst = create_engine("postgresql+psycopg2://USER:PASS@HOST:5432/db_predictive_policing")
df.to_sql(
 "minority_incidents",
 dst,
 if_exists="replace", # drops the old table first
 index=False, # do not write the pandas index column
 chunksize=500, # insert in batches of 500 rows
 method="multi", # one INSERT statement per chunk, not per row
)
```

The `chunksize=500, method="multi"` combination is a 50x speedup over default behaviour (which inserts one row at a time over the network). With 4,110 rows the difference is 2 minutes vs 2 seconds.

Why we chose rule-based filtering and not a text classifier

This is a critical design choice. Let us compare four alternatives:

Approach	Pro	Con	Verdict
Rule-based (what we picked)	Auditable, fast, no training data needed	Misses cases that use unusual phrasing	Right for this scale (4,110 rows)
Train a text classifier	Could catch unusual phrasing	Needs labelled training set we do not have	Punt for now
Embedding + cosine similarity	No labels needed	Hard to explain to PSCA auditors	Overkill
LLM zero-shot classifier (Claude/GPT)	Very high recall	Costs money per call, latency, data leaves the building	No, given the PII

The auditability point is the one that decides it. A community advocate can read our SQL and respond "those keywords are reasonable" or "you missed *Kalash* community". They cannot do that for a learned model whose decisions live in a 768-dim embedding space.

Worked example — one row through Stage 1

Suppose your raw input is this single row from `<source_call_log_table>`:

```
call_id : 8472913
level1 : Crime
<category_lvl2> : Religious Offences
description : The Christian family in our street received threats
 from neighbours after Sunday service.
 Contact: 0312-1234567
latitude : 31.5204
longitude : 74.3587
district : Lahore
tehsil : Lahore Cantt
police_station : Mughalpura
call_received_dt : 2025-12-25 14:30:00 (as a text string)
```

Step 1.4 reads this row. The filter passes (= `'Religious Offences'`). The strict label is computed: both conditions hold, so `is_minority_targeted = 1`. The date parser turns the text `"2025-12-25 14:30:00"` into a Python `datetime` object. The cleaned row is written into `minority_incidents`.

That is now the canonical record for this call. Every downstream step will use it.

Stage 2 — Look at the data before you model it (Exploratory Data Analysis)

Goal of this stage

Before training any model, **look at the data**. Plot it. Count it. Group it by district and by month. Spot the patterns that the ML model will later have to learn — and also spot the data-quality problems that will derail it if you ignore them.

In ML lingo this is **EDA — Exploratory Data Analysis**.

What "EDA" actually means

The point of EDA isn't to find the answer to your research question; it's to make sure you **understand what you are looking at** before modelling.

EDA has four loose phases:

1. **Univariate**. Look at each column in isolation. What is its mean, median, range, distribution shape?
2. **Bivariate**. Look at pairs of columns. Are they correlated?
3. **Temporal**. If there is a date column, plot every numerical column over time. Look for trends, seasonality, regime shifts, outliers.
4. **Spatial**. If there are lat/lon columns, plot them on a map.

You do not need fancy tools for any of this. Pandas, matplotlib, and seaborn cover 95% of EDA. We never used anything beyond those three.

Concept primer — descriptive statistics

Before any plot you should know these five summary numbers for any numerical column. They take 1 line of code to compute.

```
df["response_minutes"].describe()
```

Returns:

- **count** — non-null rows
- **mean** — average
- **std** — standard deviation (spread)
- **min, 25%, 50%, 75%, max** — five-number summary

What each tells you:

Statistic	What it tells you
count vs total rows	how many nulls you have
mean vs **median** (50%)	if they differ a lot, the distribution is skewed; the median is more robust
std	how widely values spread; large std = noisy
min and **max**	sanity check; if min is negative for "response time", you have a data bug

When **mean is much bigger than median** the distribution is **right-skewed** — a few very large values dragging the mean up. This is extremely common in count data (most PSes have few calls; a handful have many).

Concept primer — histograms

A **histogram** sorts your values into buckets ("0-5 calls", "6-10 calls", "11-15 calls", …) and plots the count of values in each bucket as a bar. It shows you the *shape* of the distribution.

```
import matplotlib.pyplot as plt
df["ps_count_30d"].hist(bins=30)
plt.xlabel("calls in last 30 days for the PS")
plt.ylabel("number of incidents")
plt.savefig("<see project figures>")
```

Things to look for in a histogram:

- **Unimodal** (one peak) vs **bimodal** (two peaks; suggests two underlying populations)
- **Heavy right tail** (long thin extension to the right; common in count data, suggests log-scale would help)
- **Spikes at integer boundaries** (suggests rounding or capping in the upstream system)

Concept primer — box plots

A **box plot** summarises a distribution by showing the median, the 25th-75th percentile range (the "box"), and outliers (dots beyond the "whiskers").

```
import seaborn as sns
sns.boxplot(data=df, x="community", y="response_minutes")
plt.savefig("<see project figures>")
```

Why we like box plots more than histograms for **comparing groups**: you can fit 6 box plots side by side. You cannot easily compare 6 histograms.

Concept primer — the IQR outlier rule

IQR is the "Interquartile Range" — the gap between the 25th and 75th percentile. A common rule of thumb says a value is an *outlier* if it sits more than 1.5 IQRs above the 75th percentile or more than 1.5 IQRs below the 25th. This is the rule a default box plot uses.

```
q1, q3 = df["response_minutes"].quantile([0.25, 0.75])
iqr = q3 - q1
mask = (df["response_minutes"] < q1 - 1.5*iqr) | (df["response_minutes"] > q3 + 1.5*iqr)
outliers = df[mask]
```

We used this rule to flag the **24 rows with lat/lon outside Pakistan entirely** as outliers. Those rows are filtered out at dashboard build time, not deleted from the data table (so we keep an audit trail).

Concept primer — Pearson vs Spearman correlation

To check if two numerical columns vary together:

- **Pearson** measures *linear* correlation. Returns a value between -1 (perfect inverse) and +1 (perfect positive). 0 means no linear relationship.
- **Spearman** measures *monotonic* correlation. Better when the relationship is non-linear but still consistent (e.g. one always goes up as the other goes up, but not in a straight line).

```
df[["ps_count_30d", "ps_count_60d", "ps_count_90d"]].corr(method="pearson")
```

We found `ps_count_30d` and `ps_count_60d` correlated at 0.93 — they carry essentially the same information. We kept both because the model can decide which window to weight more, but we noted this in the model card so a future reader knows the features are not independent.

Concept primer — time series resampling

If your data has a date column, you almost always want to **resample** it to a regular frequency.

```
df.set_index("incident_date", inplace=True)
monthly = df.resample("M").size() # number of rows per month
weekly = df.resample("W").size() # per week
daily = df.resample("D").size() # per day
"M"` means "month-end". `"W"` means "week-end (Sunday)". `"D"` means
"day". Pandas has dozens of these codes. The result is a Series
indexed by date.

## Concept primer - regime shifts and changepoint detection

A **regime shift** is when a time series suddenly changes its
behaviour at a single point. Before the point the series looks one
way; after, it looks different (different mean, different variance,
or both). They are very common in real data because the **upstream
process** changed - software was redeployed, categories were
re-tagged, a policy changed.

Eyeballing a plot is usually enough to spot a regime shift if you
```

```
know to look. Statistical tests exist (Bai-Perron, PELT, CUSUM) but
we did not need them. The Oct 2025 drop was visible to the naked eye.

**This is the single most important finding of our EDA.** Without it
we would have trained Prophet on the full timeseries and gotten a
forecast that was 100% off (we did, in fact, do exactly that on the
first attempt - MAPE was 103.81%).

## What we did

### Step 2.1 - Basic counts

```python
df = pd.read_sql("SELECT * FROM minority_incidents", dst)
print(df["district"].value_counts().head(10))
print(df["community_named"].value_counts())
print(df["<category_lvl2>"].value_counts())
```

Findings:

- Lahore: 928 rows (22.6% of all minority cases)
- Christian: most-named community (~58% of named-community rows)
- Ahmadi: ~17%
- Hindu: ~14%
- Sikh: ~7%
- `<category_lvl2> = "Religious Offences"` covers 38% of our 4,110 — meaning the keyword-based rules pull in another 62%

Step 2.2 — Plot calls over time

```
ts = df.groupby(df["incident_date"].dt.to_period("M")).size()
ts.plot(kind="bar", figsize=(14,4))
plt.title("Minority-related calls per month")
plt.savefig("<see project figures>")
```

The plot revealed the October 2025 regime shift. Before Oct 2025 the monthly volume sat around 200; from Oct 2025 it dropped to ~90.

Step 2.3 — Plot calls by district

A simple bar chart showed Lahore dominating the dataset. This becomes important when interpreting forecaster output: top-50 risk PSes are heavy with Lahore PSes, not because Lahore is uniquely violent but because Lahore is over-represented in training.

Step 2.4 — Plot calls relative to religious holidays

We marked Christmas, Eid, Holi, Diwali, Muharram on a calendar, then for each holiday we plotted **days-relative-to-holiday vs call count**.

```
for holiday_name, holiday_dates in HOLIDAYS.items():
 df["days_to"] = df["incident_date"].apply(
 lambda d: min((pd.Timestamp(h) - d).days for h in holiday_dates)
 )
```
```

```
bucket = df.groupby("days_to").size()
bucket.plot()
```

Finding: 18.9% of minority-related calls fall within +/- 1 day of a major religious holiday. Christmas Day 2025 alone saw a 5-district spike.

Step 2.5 — Plot a Lahore-only spatial heatmap

We loaded Leaflet's heatmap layer with every Lahore call's lat/lon. Visible hotspots emerged:

- Mughalpura
- Garhi Shahu
- Chung
- Kahna

These are our four most-mentioned PSes in the strict-label subset.

Why EDA matters in practice — three real bugs caught

Without EDA we would have shipped three bugs into production:

1. **Trained Prophet on the full timeseries** including the Oct 2025 regime shift. MAPE was 103.81%. After restricting to the post- shift window, MAPE dropped to 9.54%.
2. **Trusted the keyword filter blindly** without realising 38% of cases come purely from the `<category_lvl2>` tag. This affected how we wrote up Stage 1's design choice in the policy brief.
3. **Missed the Christmas 2025 spike**, which became one of the strongest empirical findings in the policy brief.

EDA is also the single phase where bad data is most cheaply caught. Once a bad row is propagated into features and then into a model, diagnosing the root cause becomes exponentially harder.

Stage 3 — Build "features" so the model has something to learn from

Goal of this stage

A raw phone call has things like `call_id`, `district`, `latitude`, `description`. None of those are *predictive features* the way a model wants. We have to **construct** the features that the model will actually look at.

We grouped features into **six families**.

Concept primer — what is a "feature"?

A **feature** is a column of numbers (or a one-hot-encoded category) that captures one piece of information about the row. A model takes in a vector of features for each row and tries to learn how those features predict the target.

Example: if you are predicting whether a student will pass an exam, useful features might be:

- hours studied this week
- average grade in previous semester
- whether the student is in study group X (one-hot)
- days since last extra-credit assignment

We do the same thing for incidents. For every phone call we want to predict on, we attach numbers like:

- how many calls came from this same PS in the last 30 days?
- days until next Christmas?
- the local PS's median response time over the last 3 months?

Concept primer — feature types

There are five basic feature types you will work with:

Type	Example	How to feed into a model
Numerical (continuous)	response time = 14.7 minutes	Pass directly; sometimes scale to mean 0 std 1
Numerical (count)	`ps_count_30d` = 7	Pass directly; consider log1p if heavy-tailed
Categorical (nominal)	`community` in {Christian, Hindu, …}	**One-hot encode** — create a 0/1 column per category
Categorical (ordinal)	"low / medium / high"	Map to integers 0/1/2 OR one-hot
Datetime	`incident_date = 2025-12-25 14:30`	Decompose into year, month, day-of-week, hour, then engineer time-since-event features
Text	description (free text)	Bag-of-words, TF-IDF, embeddings, or hand-coded keyword flags

We made all five types appear in our feature table.

Concept primer — what is "feature engineering"?

The art of designing those features. Done well, it can lift a model from useless to good. Done badly, the model has nothing to learn.

In modern deep learning (image classifiers, big language models) features are *learned automatically*. But for tabular data with 4,000 rows like ours, **hand-engineered features dominate**. There simply is not enough data for a deep network to discover them on its own.

A good feature has three properties:

1. **Predictive** — it correlates with the target.
2. **Cheap to compute** — fast enough to recompute weekly.
3. **Available at prediction time** — you can compute it without peeking at the future.

Property 3 is the one beginners always violate. It is called the **no-look-ahead** rule.

Concept primer — rolling-window features

A **rolling window** is a feature that summarises the past N units of time before the current row. The classic ones are:

Window aggregation	Code	What it captures
Count	`.rolling(window).count()`	activity level
Sum	`.rolling(window).sum()`	total magnitude

Window aggregation	Code	What it captures
Mean	`.rolling(window).mean()`	typical level
Std	`.rolling(window).std()`	volatility
Max	`.rolling(window).max()`	recent peak
Min	`.rolling(window).min()`	recent low

Choosing N (the window length) is itself a design choice. Common values are 7, 14, 30, 60, 90 days. We picked 30, 60, 90 because they correspond to one, two, three months — easy for stakeholders to interpret.

Concept primer — the haversine formula

When you want the great-circle distance between two latitude / longitude points (i.e. distance along the curved earth surface, not straight-line through the planet), you use the **haversine** formula:

```
a = sin^2((lat2 - lat1)/2) + cos(lat1) * cos(lat2) * sin^2((lon2 - lon1)/2)
c = 2 * atan2(sqrt(a), sqrt(1-a))
d = R * c # R = 6371 km, Earth's mean radius
```

We use this when computing "PS-level density" features that need to know which incidents fall within X meters of each other. In Python:

```python
from math import radians, sin, cos, sqrt, atan2

def haversine_m(lat1, lon1, lat2, lon2):
 R = 6371000 # metres
 phi1, phi2 = radians(lat1), radians(lat2)
 dphi = radians(lat2 - lat1)
 dl = radians(lon2 - lon1)
 a = sin(dphi/2)**2 + cos(phi1)*cos(phi2)*sin(dl/2)**2
 return 2 * R * atan2(sqrt(a), sqrt(1-a))
```

For larger datasets, `geopy.distance.geodesic` is more accurate (uses the WGS-84 ellipsoid), but haversine is fast and accurate enough at Punjab scale.

Concept primer — cyclic encoding for dates

If you put `month_of_year` (1 to 12) into a model as a single numerical feature, the model thinks December (12) is far from January (1). But December and January are *adjacent* months. To handle this, encode with sin/cos:

```python
df["month_sin"] = np.sin(2 * np.pi * df["incident_date"].dt.month / 12)
df["month_cos"] = np.cos(2 * np.pi * df["incident_date"].dt.month / 12)
```

These two columns together encode the *cyclical* nature of months. December's `(sin, cos)` is close to January's. We did this for month, day-of-week, and hour-of-day where it mattered.

Concept primer — one-hot encoding

For a categorical column like `community in {Christian, Hindu, Sikh, Ahmadi, Other}`, we cannot just feed a string to LR or RF. We convert it to **five binary columns**:

```
community is_Christian is_Hindu is_Sikh is_Ahmadi is_Other
Christian 1 0 0 0 0
```

```
Hindu 0 1 0 0 0
Sikh 0 0 1 0 0
...
```

pandas does this in one line:

```
df = pd.get_dummies(df, columns=["community"], prefix="is", drop_first=False)
drop_first=False` keeps all categories. If you set `drop_first=True`,
one category is dropped (avoiding multicollinearity in linear models
- a concept worth knowing but not critical here).

## Concept primer - feature scaling

Logistic regression's coefficients are sensitive to feature scale. If
one feature has values in [0, 1] and another in [0, 100000], the
coefficient on the second will be tiny just because of scale, not
because it is unimportant. To fix this, **standardise**:

```python
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

This subtracts the mean and divides by the standard deviation for each column. After scaling, every column has mean 0 and std 1.

Random Forest does NOT need scaling — splits are scale-invariant. We still scale because we run both models off the same feature matrix for code simplicity.

Crucially: fit the scaler on training data only, then apply to test data. Never fit on test data — that is information leakage from test back into training.

The six feature families — in detail

Family 1 — Prior density (history features)

> *Idea: if a place has been tense recently, it is probably still tense.*

For each row, we count how many minority-related calls came from the same PS (and the same district) in the previous 30 / 60 / 90 days:

```
def density_for_row(row, df, days):
 cutoff = row["incident_date"] - pd.Timedelta(days=days)
 mask = (
 (df["police_station"] == row["police_station"])
 & (df["incident_date"] >= cutoff)
 & (df["incident_date"] < row["incident_date"])
 )
 return mask.sum()
```

The `< row["incident_date"]` (strict less-than) is critical: we must not include the current row in its own history feature.

This gives us features like `ps_count_prev_30d`, `ps_count_prev_60d`, `ps_count_prev_90d`, plus the same three at district level.
```

Family 2 — Religious calendar

Idea: incidents cluster near major religious holidays.

For each row, compute `days_until_next_christmas`, `days_until_next_eid`, `days_until_next_muharram`, `days_until_next_holi`, `days_until_next_diwali`. Also their absolute distances (how *close* the row is to the holiday in either direction):

```
HOLIDAYS = {
 "christmas": ["2024-12-25", "2025-12-25", "2026-12-25"],
 "eid_fitr": ["2024-04-10", "2025-03-31", "2026-03-20"],
 "muharram": ["2024-07-17", "2025-07-06", "2026-06-26"],
 ...
}

def days_until(date, holiday_list):
 future = [pd.Timestamp(d) - date for d in holiday_list
 if pd.Timestamp(d) >= date]
 return min(future).days if future else None

def closest_distance(date, holiday_list):
 return min(abs((pd.Timestamp(d) - date).days) for d in holiday_list)
```

Why this is powerful: the model can learn "when `days_until_christmas < 10`, baseline risk is elevated". Without this feature the model would have to discover the holiday effect from date columns alone — much harder.

Family 3 — Political calendar

Election dates and major political announcements. Built as one-hot flags plus a `days_since_last_political_event` field.

The 2024 Pakistani general election was held on Feb 8, 2024. Calls in the surrounding weeks did show a small uptick — visible in EDA.

Family 4 — Misinformation events

A manually-curated list of viral misinformation episodes — fake blasphemy accusations, false claims about minority-owned businesses, etc. For each row we compute `days_since_last_misinfo_in_district`.

This list is small (~30 entries) and we plan to grow it once a media monitoring desk feeds in. Currently maintained as a JSON file in the repo.

Family 5 — Local police responsiveness

For the PS that handled this call, what is the **median response time** of that PS over the last 90 days? Slow PSes correlate with higher minority-incident rates (likely a vulnerability signal, not a causation signal — but the model only learns the correlation):

```
def median_response_time(ps, asof, df_response):
 cutoff = asof - pd.Timedelta(days=90)
 sub = df_response[
 (df_response["police_station"] == ps)
 & (df_response["dt"] >= cutoff)
```

```
    & (df_response["dt"] < asof)
    ]
    return sub["response_minutes"].median()
```

We use **median** rather than mean because response-time distributions are heavy-tailed (a handful of 500-minute calls would drag the mean up; the median is robust).

Family 6 — Sentiment (placeholder)

We have not yet connected a social-media feed. The column exists in the schema as NULL; once a feed is connected the model can pick it up automatically. This is an honest **placeholder** — we marked it as such in every report and we did not invent fake values.

Concept primer — handling missing values

After all that feature engineering, you will inevitably have some `NaN`s. The choices:

Strategy	When to use it
Drop rows with any NaN	Only when missingness is tiny (<1%) and random
Fill with 0	When NaN literally means "no events", e.g. `ps_count_30d` for a brand-new PS
Fill with median	When NaN is true missingness on a numerical column
Fill with "Missing" category	For categoricals; can be informative on its own
Model-based imputation (KNN, IterativeImputer)	For high-stakes cases where missingness correlates with the target

We mostly used 0-filling for count features (since NaN means "no history exists") and median for response-time features. We never silently dropped rows.

Why these six families and not others?

Alternative family	Why we did not include it
Caller demographics (gender, age)	Not in the raw data
Officer-level features (which officer responded)	Risky from a fairness perspective
Embedding of the description text (BERT, etc.)	Overkill on 4,110 rows. We would risk overfitting on text that does not generalise
Weather (heatwave, rain)	Considered it; couldn't find a free clean API for Punjab fine-grained enough
Macroeconomic indicators (CPI, unemployment)	Too low-frequency for our weekly forecast
Network features (who-knows-whom)	Would need social graph data we do not have

The six we picked cover **time, place, context, calendar, police capacity, and a placeholder for media** — and that is already enough.

Worked example — features attached to one row

Take call_id 8472913 (Christmas Day 2025, Mughalpura, Christian family). After feature engineering it looks like:

```
call_id : 8472913
ps_count_prev_30d : 7 (Mughalpura has been busy)
ps_count_prev_60d : 12
ps_count_prev_90d : 21
district_count_30d : 41 (Lahore-wide last 30 days)
days_until_christmas : 0 (today IS Christmas)
days_until_eid_fitr : 96
days_until_muharram : 197
days_since_last_misinfo_district : 3
median_resp_time_ps_90d : 14.7 (minutes)
days_since_last_election : 687
month_sin : -0.50
month_cos : 0.87
is_Christian : 1
is_Hindu : 0
is_Sikh : 0
is_Ahmadi : 0
is_minority_targeted : 1 (label - what we are predicting)
```

The model will learn that *days_until_christmas = 0 AND ps_count_prev_30d = 7* together push the predicted probability up.

Stage 4 — Reshape the data for forecasting (the PS-week table)

Goal of this stage

The per-case features answer *"is this call a minority attack?"* — a **classification** question. We also want to answer *"in the next 30 days, will there be a minority attack in police-station X?"* — a **forecasting** question.

These are two different question shapes, so they need two different training tables.

Concept primer — classification vs forecasting

	Classification	Forecasting
What is one row?	One observed thing (a phone call)	One time window for one entity (PS x week)
What is the label?	Property of that thing	Did the event occur in the *future* window?
What does the model output?	Score for each input row	Score for each future window
Where is "time" in the input?	A feature, but often only as a snapshot	The data is indexed by time; features describe the past, label describes the future
Cross-validation?	Random k-fold is fine if rows are independent	Must use **temporal** cross-validation

Classification is "given this thing right now, label it". Forecasting is "given the past of this entity, predict its future".

Concept primer — panel data

A table where you have many entities observed at many points in time is called **panel data** (or "longitudinal data" in social science). Examples:

- countries x years (GDP growth)
- stores x weeks (sales)
- patients x doctor-visits (vital signs)
- **police-stations x weeks (minority incident risk)** — our case

The standard shape is `(entity_id, time_period, ...features..., label)`. Each row is one entity at one time period.

How we build the PS-week table

For every (police station, week) combination, we create one row:

```
ps_name | week_start | label_any_minority_incident_in_next_30d | features ...
Mughalpura | 2024-01-01 | 1 | ...
Mughalpura | 2024-01-08 | 0 | ...
Mughalpura | 2024-01-15 | 1 | ...
...
Sadar (Lahore) | 2024-01-01 | 0 | ...
...
```

With 656 PSes x ~120 weeks of training data, we get about **82,000 rows**.

The label for row `(PS=X, week=W)` is:

```
1 if any minority_incident occurred in PS X during the 30 days
 following the start of week W
0 otherwise
```

This is the **forecasting target**: did a minority incident happen in this PS's *future* 30 days?

Concept primer — temporal leakage (the cardinal sin)

The single biggest pitfall in time-series modelling is **temporal leakage**: a feature accidentally uses information from after the moment of prediction. This will silently inflate your model's test score and then crash in production when you cannot magically know the future.

Examples of leakage we explicitly avoided:

- Using "calls in next 30 days" as a feature (that *is* the label!)
- Using "average response time across the whole dataset" — that includes the future and contaminates older rows
- Using "is the call in a holiday month" without checking the holiday date is in the past relative to the row's date — for "days until next Christmas", this is fine; for "days since last Christmas", be careful

Our rule: **for every (PS, week) row, all features must be computable using only data from before `week_start`. Period.**

Concretely:

- The label uses the *next* 30 days (that is fine — the label is what we are predicting, by design).
- The features use the *prior* 30/60/90 days.

- There is **no overlap** between the windows used for label and for features.

To enforce this in code we wrote a `compute_features_asof(ps, asof_dt)` function that takes a "cutoff datetime" and refuses to look past it. Every feature query uses it.

Why we picked weekly granularity

Alternative	Why we did not pick it
Daily	Too sparse — most (PS, day) pairs have zero minority calls, label is 0 almost everywhere. Model learns nothing
Monthly	Too coarse — by the time we predict next month, the warning is half-useless
Bi-weekly	Reasonable, but odd to explain ("the model gives risk for the next 14 days starting Monday")
Hourly	Way too sparse; also operationally meaningless

Weekly is the **smallest unit where the label has reasonable variance** while still giving an operationally useful lead time.

Worked example

For row `(PS=Mughalpura, week_start=2025-12-22)`:

- Label: is there a minority incident in Mughalpura between 2025-12-22 and 2026-01-21? **Yes** (Christmas Day 2025 incident), so label = 1.
- Features (all computed as of 2025-12-21 23:59):
- `ps_calls_30d_before` = 7
- `district_calls_30d_before` = 41
- `days_until_christmas` = 3
- `median_resp_time_ps_90d` = 14.7

That is a "this PS will see a minority incident in the next 30 days" training example.

Stage 5 — Train the per-case classifier (Logistic Regression + Random Forest)

Goal of this stage

Given a single new phone call, output a probability that it is a real minority-targeted incident. We train two models: a Logistic Regression (LR) and a Random Forest (RF). We compare them.

Concept primer — supervised learning

Supervised learning is the family of ML where you have:

- a bunch of input rows X (features)
- a bunch of labels y (what you want to predict)
- the goal: learn a function `f` such that `f(X) ~= y`

After training, you apply `f` to new rows where you do not know y, and treat `f(X_new)` as the prediction.

Supervised learning splits into two big sub-categories:

- **Classification** — y is a category (0 or 1, or "spam"/"not spam", or "cat"/"dog"/"horse")
- **Regression** — y is a number (price, response-time-in-minutes)

We do **binary classification** here: y is 0 or 1.

Concept primer — Logistic Regression in detail

Logistic regression sounds scary; the idea is simple.

The model equation

> Imagine every feature gets a **weight**. To score a row, take each feature value, multiply by its weight, sum them all up, plus an intercept, and push the sum through a curve that maps it to a number between 0 and 1. That number is the predicted probability that the label is 1.

Mathematically, for row `i`:

```
z_i = b + w1 * x1_i + w2 * x2_i + ... + wn * xn_i
p_i = sigmoid(z_i) = 1 / (1 + exp(-z_i))
```

Where:

- `b` is the intercept (a single number)
- `w1, w2, ..., wn` are the weights (one per feature)
- `x1_i, x2_i, ..., xn_i` are the feature values for row `i`
- `z_i` is the **logit** or **log-odds**
- `p_i` is the **predicted probability**

The **sigmoid function** maps any real number to (0, 1):

```
sigmoid(0) = 0.5
sigmoid(very large positive) ~ 1
sigmoid(very large negative) ~ 0
```

It is the smooth S-shape that turns "raw scores" into probabilities.

Interpreting the weights — log-odds and odds ratios

The natural interpretation of LR weights is in **log-odds**. If `w_j` is the weight on feature `j`:

- Increasing `x_j` by 1 unit increases the log-odds of the positive class by `w_j`.
- Equivalently, multiplies the **odds** by `exp(w_j)`.

So if our LR learned `w[days_until_christmas] = -0.05`:

- Each additional day away from Christmas multiplies the odds of being minority-targeted by `exp(-0.05) = 0.95`.
- Ten days further away: `exp(-0.5) = 0.61` (about 40% lower odds).

This is the property that makes LR **interpretable**. You can read the weights and say what changes the prediction by how much. This is critical for stakeholder trust.

How LR learns the weights

We use **Maximum Likelihood Estimation (MLE)**: choose weights that make the observed labels *most probable* under the model. For binary classification this is equivalent to minimising the **cross-entropy loss** (also called log-loss):

```
loss = -1/N * sum_i [ y_i * log(p_i) + (1 - y_i) * log(1 - p_i) ]
```

This loss is differentiable, so we can use **gradient descent** (or its variants L-BFGS, SAG, SAGA) to find the optimal weights. scikit-learn picks the solver automatically based on the problem size; we let it pick.

Regularisation — L1 vs L2

Even with 30 features and 4,000 rows, weights can grow large and overfit. We add a **regularisation term** to the loss:

- **L2** (Ridge) adds `lambda * sum(w_j^2)` to the loss. Shrinks weights but never to exactly zero.
- **L1** (Lasso) adds `lambda * sum(|w_j|)` to the loss. Encourages some weights to be exactly zero (effectively does feature selection).

`lambda` is the regularisation strength. In scikit-learn it is expressed as `C = 1 / lambda` — larger `C` means weaker regularisation. We used the default `C=1.0` because we did not have enough data for a proper grid search to be meaningful.

We used **L2** because we wanted to keep all features (interpretation is easier when no weights are zero).

Class imbalance handling — the math

Only 312 / 4110 = 7.6% of rows are positive. Without intervention, the model would heavily favour predicting 0 (because most rows are 0). The standard fix is **class weights**: weight each rows loss contribution by the inverse class frequency.

```
n_positive = 312
n_negative = 3798
n_total = 4110

w_positive = n_total / (2 * n_positive) = 6.59
w_negative = n_total / (2 * n_negative) = 0.54
```

So each positive row counts as 6.59 rows in the loss, each negative counts as 0.54. The model is now incentivised to get positives right.

scikit-learn's `class_weight='balanced'` does this automatically:

```
lr = LogisticRegression(class_weight="balanced", max_iter=1000)
```

When LR will not be enough

LR assumes a **linear** relationship between each feature and the log-odds of the label. If the true pattern is non-linear (e.g. risk is high when `ps_count_30d` is BOTH very low and very high but low in the middle), LR cannot capture it without manual feature engineering.

That is where Random Forest comes in.

Concept primer — Random Forest in detail

Step 1 — what is a decision tree?

A **decision tree** is a flowchart of yes/no questions. At each node, the algorithm picks the feature and threshold that best separates the data into two groups. At each leaf, it predicts the majority class (or, for probability output, the proportion of positives in that leaf).

Example tiny tree:

```
ps_count_30d > 5?
/ \
No Yes
| |
days_until_christmas label = 1 (probability 0.85)
< 7?
/ \
No Yes
| |
label = 0 label = 1
(p=0.05) (p=0.60)
```

Three terminology terms:

- **Node** — a yes/no question
- **Leaf** — a terminal node (no further questions)
- **Depth** — how many yes/no questions you can be asked along the longest path

Step 2 — how does the tree pick splits?

For each candidate (feature, threshold) split, the tree computes how **pure** the two resulting groups are. The most common purity measures are:

Gini impurity:

`gini = 1 - sum_k (p_k)^2`

where `p_k` is the proportion of class `k` in the node. For binary problems, `gini = 1 - p^2 - (1-p)^2 = 2*p*(1-p)`. Range: 0 (pure) to 0.5 (50/50 mix).

Entropy:

`entropy = -sum_k p_k * log2(p_k)`

Range: 0 (pure) to 1 (50/50 mix).

The tree picks the split that maximises **information gain** — parent's impurity minus the weighted average of children's impurity:

```
gain = impurity(parent) - (n_left/n_total * impurity(left)
 + n_right/n_total * impurity(right))
```

scikit-learn defaults to Gini. Both Gini and Entropy give similar results in practice; we used Gini.

Step 3 — when does the tree stop splitting?

Three hyperparameters control depth:

Hyperparameter	What it does	Our value
`max_depth`	max number of yes/no questions in a path	8
`min_samples_split`	minimum rows required to consider splitting a node	2 (default)
`min_samples_leaf`	minimum rows in a leaf	1 (default)

We picked `max_depth=8` because deeper trees overfit on our small dataset (4,110 rows). 8 means at most 256 leaves — about 16 rows per leaf on average. Reasonable.

Step 4 — from a single tree to a forest

A single decision tree is **high-variance** — change a few training rows and you can get a wildly different tree. The fix is **bagging** (bootstrap aggregating):

1. Draw N random samples *with replacement* from the training data. Each draw is the same size as the original. About 63% of original rows appear in any given draw; 37% are left out.
2. Train one tree on each draw.
3. For prediction, average the trees' predicted probabilities.

A **random forest** also adds **feature bagging**: at each split, only consider a random subset of features (usually sqrt of total features for classification). This decorrelates the trees and improves ensemble performance.

We used **300 trees**:

```
rf = RandomForestClassifier(
 n_estimators=300,
 max_depth=8,
 class_weight="balanced",
 random_state=42,
 n_jobs=-1, # use all CPU cores
)
```

Step 5 — feature importance from a Random Forest

A nice side effect of RF: it gives you a feature-importance ranking. The most common measure is **Mean Decrease in Impurity (MDI)**:

```
importance(feature j) = average over trees of
 (sum over nodes splitting on j of
 (impurity_drop_at_that_node * fraction_of_data_at_node))
```

In practice you just look at `rf.feature_importances_`:

```
import pandas as pd
imp = pd.Series(rf.feature_importances_, index=feature_names)
imp.sort_values(ascending=False).head(10)
```

We got (top 5):

Feature	RF importance
`<category_lvl2> == 'Religious Offences'` (one-hot)	0.34 (dominant)
`median_resp_time_ps_90d`	0.09
`ps_count_prev_90d`	0.07
`days_until_christmas`	0.05
`district_count_prev_30d`	0.04

Caveat: MDI is biased toward high-cardinality numerical features. For a more robust ranking, use **permutation importance**:

```
from sklearn.inspection import permutation_importance
result = permutation_importance(rf, X_test, y_test, n_repeats=10, n_jobs=-1)
```

Permutation importance shuffles each column in turn and measures how much the model's score drops. We checked both; the top-5 list above held up under both rankings.

When RF will not be enough either

- Harder to interpret than LR.
- Can overfit on small datasets if you do not cap `max_depth`.
- Does not naturally **extrapolate** beyond the training range. If the training data only saw `ps_count_30d` up to 50 and a new row has `ps_count_30d=200`, the forest treats it the same as 50.

Concept primer — train/test split, the right way

For our project the rule was:

```
train = features[features["incident_date"] < "2026-01-01"]
test = features[features["incident_date"] >= "2026-01-01"]
```

Train on rows from 2024 and 2025; test on rows from 2026.

Why temporal split and not random? Because in production the model predicts on data from *after* the training window. A random split lets information leak from late 2025 into the training and gives a falsely optimistic test score.

If we had more data we would also use **walk-forward cross-validation** (also called expanding window):

```
Fold 1: train on Jan-Jun 2024, test on Jul 2024
Fold 2: train on Jan-Jul 2024, test on Aug 2024
Fold 3: train on Jan-Aug 2024, test on Sep 2024
...
```

But with 4,110 rows total and a 312-positive class, 12 folds would give us roughly 26 positives per fold — too few to be statistically meaningful. We skipped this.

Concept primer — evaluation metrics

For a binary classifier, the workhorse metrics are:

Confusion matrix:

	Predicted 0	Predicted 1
Actual 0	TN	FP
Actual 1	FN	TP

From these four counts you can compute:

- **Precision** = TP / (TP + FP) — of cases the model called 1, what fraction actually are 1
- **Recall** = TP / (TP + FN) — of cases that actually are 1, what fraction did the model catch
- **F1** = harmonic mean of precision and recall
- **Accuracy** = (TP + TN) / total — fraction correctly classified; *meaningless when classes are imbalanced*

ROC curve: sweep the threshold from 0 to 1; for each threshold plot true-positive rate (recall) on Y-axis and false-positive rate on X-axis. The area under this curve is the **ROC AUC**:

- AUC = 0.5: random
- AUC = 0.7: usable
- AUC = 0.8: good
- AUC = 0.9: excellent (but check for leakage)
- AUC = 1.0: either perfect or, more likely, you have leaked the label

Precision-Recall curve: sweep threshold; plot precision (Y) vs recall (X). Better than ROC for very imbalanced problems because it ignores the (vast) TN count.

We report ROC AUC, plus precision and recall at threshold 0.5, plus precision and recall at the threshold that maximises F1 on the validation set.

What we did, step by step

Step 5.1 — Train/test split

Already covered above.

Step 5.2 — Scaling

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
joblib.dump(scaler, "<see project models>")
```

We saved the scaler so prediction can re-apply the same transformation.

Step 5.3 — Train LR

```
from sklearn.linear_model import LogisticRegression
lr = LogisticRegression(
 class_weight="balanced",
 max_iter=1000,
 solver="lbfgs",
 random_state=42,
)
```

```
lr.fit(X_train_scaled, y_train)
joblib.dump(lr, "<see project models>")
```

Step 5.4 — Train RF

```
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(
 n_estimators=300,
 max_depth=8,
 class_weight="balanced",
 n_jobs=-1,
 random_state=42,
)
rf.fit(X_train_scaled, y_train)
joblib.dump(rf, "<see project models>")
```

Step 5.5 — Evaluate on test data

```
from sklearn.metrics import (
 roc_auc_score, precision_score, recall_score, f1_score,
 classification_report, confusion_matrix
)

p_lr = lr.predict_proba(X_test_scaled)[:, 1]
p_rf = rf.predict_proba(X_test_scaled)[:, 1]

print("LR AUC:", roc_auc_score(y_test, p_lr)) # 0.84
print("RF AUC:", roc_auc_score(y_test, p_rf)) # 0.81

threshold = 0.5
y_pred_lr = (p_lr >= threshold).astype(int)
print(classification_report(y_test, y_pred_lr))
```

Step 5.6 — Write a model card

Every trained model writes a `model_card.md` describing what it was trained on, what its metrics are, what its limitations are. This is non-negotiable for any model that will be presented externally.

Why we did LR + RF and not just one

Different models give different strengths:

- **LR for interpretability**: when you present this to PSCA management or to a community advocate, they want to see "increasing `days_until_christmas` decreases risk; increasing `ps_count_30d` increases risk". LR makes that easy.
- **RF for non-linearities**: maybe `ps_count_30d` only matters when `days_until_christmas` is small. RF can learn that automatically; LR cannot without us adding interaction terms.

The LR's slightly higher AUC (0.84 vs 0.81) means it is actually doing *better* than RF on this dataset — which sometimes happens with small clean tabular data. We kept both because the dashboard displays the RF probability prominently (RF gives slightly more *conservative* probabilities, which is desirable for triage).

Why not other models?

Alternative	Why we did not pick it
XGBoost / LightGBM	Marginally better than RF, more hyperparameters to tune, no real lift on 4,110 rows
Neural net	Needs much more data, harder to interpret, no benefit here
SVM with RBF kernel	Slow to train at scale, harder to interpret
Naive Bayes	Too simple, ignores correlations between features
Catboost	Overkill; comparable to LightGBM in our regime

The general principle: **start with the simplest model that does the job, escalate only if it fails**. On 4,110 rows with 30-ish features, LR and RF are the right zone.

Worked example

For call_id 8472913 (Christmas Day, Mughalpura), the per-case model output:

- LR probability of "minority-targeted" = **0.87**
- RF probability of "minority-targeted" = **0.94**

Both above 0.5, both say "yes this looks like a real one". The triage team gets to see this in the dashboard.

Stage 6 — Train the PS-week forecaster (the actual "early warning")

Goal of this stage

For every police-station in Punjab, output a probability that a minority incident will happen there in the next 30 days. This is the **operational** model — it tells field commanders where to deploy extra patrols *before* an incident.

How is this different from Stage 5?

Per-case (Stage 5)	PS-week (Stage 6)
Input: one phone call	Input: one (PS, week) row
Output: probability this call is real	Probability of any minority incident in this PS in next 30 days
When do you score? After every new call	Once a week for every PS
Use case	Triage incoming calls

The features are similar in spirit but reshaped to the (PS, week) grain (Stage 4).

What we did

We used the same Logistic Regression + Random Forest pair, fit on the **PS-week training table** from Stage 4. We chose LR as the deployed forecaster because Random Forest collapses on the sparse positive class (it pushes almost no mass to "yes"), while LR ranks the police stations far better.

Calibration caveat. LR ranks well, but its *raw* output is **not a well-calibrated probability** at this grain: because the model is trained with balanced class weights on a ~0.75%-rate event, it emits an average "probability" near 0.58. Treat the forecaster's number as a **risk ranking, not a literal probability**. A monotonic re-fit (isotonic regression) brings the scores back in line with observed frequencies (calibration error drops ~30×) if a true probability is ever needed — but it does not change the ranking, so the top-N early-warning list is unaffected.

Train

```
ps_train = pd.read_csv("<see project intermediate data>")
ps_test = pd.read_csv("<see project intermediate data>")

X_train = ps_train.drop(columns=["label"])
y_train = ps_train["label"]
X_test = ps_test.drop(columns=["label"])
y_test = ps_test["label"]

scaler = StandardScaler().fit(X_train)
X_train_s = scaler.transform(X_train)
X_test_s = scaler.transform(X_test)

lr = LogisticRegression(class_weight="balanced", max_iter=2000)
lr.fit(X_train_s, y_train)
```

Evaluate by ranking — top-K precision and lift

For a forecaster like this, what we really care about is **ranking**. If we tell PSCA "patrol the top-50 highest-risk PSes this week", we want as many of those 50 to be **real** at-risk PSes as possible.

So we compute:

- **Top-K precision** = (true positives in top-K) / K
- **Top-K lift** = top-K precision / base rate of positives

```
import numpy as np
p_test = lr.predict_proba(X_test_s)[:, 1]
order = np.argsort(p_test)[::-1] # indices sorted by score desc

k = 50
top_k_idx = order[:k]
top_k_precision = y_test.iloc[top_k_idx].mean()
base_rate = y_test.mean()
lift = top_k_precision / base_rate

print(f"Top-{k} precision = {top_k_precision:.3f}")
print(f"Base rate = {base_rate:.3f}")
print(f"Lift = {lift:.1f}x")
```

Our forecaster's top-50 had a **lift of ~18x** over random guessing. That means if the base rate of "PS has at least one minority incident in next 30 days" is about 1.5% of all PS-weeks, our top-50 captured them at about 27% — about 18 times better than chance.

Concept primer — calibration

A classifier is **well-calibrated** if among all the rows it gives score `p`, exactly `p` fraction are actually positive. So if we look at all rows scored ~0.3 they should be positive about 30% of the time; all rows scored ~0.8 should be positive about 80% of the time.

To check calibration, bin predicted scores and plot bin midpoint vs observed positive rate:

```
from sklearn.calibration import calibration_curve
prob_true, prob_pred = calibration_curve(y_test, p_test, n_bins=10)
plt.plot(prob_pred, prob_true, marker="o")
plt.plot([0,1], [0,1], "--", label="perfect")
```

If your model is poorly calibrated, you can fix it with:

- **Platt scaling** — fit a logistic regression on the raw scores
- **Isotonic regression** — fit a monotonic step function on raw scores

We did not need calibration fixing because LR is naturally calibrated when trained with log-loss.

Generate forward predictions

Once the model is trained, we compute features for every PS *as of today* and predict 30 days forward:

```
today = pd.Timestamp.today()
forward = compute_features_for_all_ps_asof(today)
forward["risk_30d"] = lr.predict_proba(scaler.transform(forward[FEATS]))[:, 1]
forward.sort_values("risk_30d", ascending=False).to_csv(
 "<see project intermediate data>", index=False
)
```

This produces a 656-row table — one per PS — sorted by `risk_30d` descending.

Why we picked Logistic Regression for the forecaster

The alternative we *seriously* considered was a **Hawkes process** (also called a "self-exciting point process"), which is the classical model for events that cluster in time and space (earthquakes, terror incidents, financial trades).

A Hawkes process treats the event intensity as the sum of:

- a baseline rate per location, and
- a self-exciting term: every past event raises the local intensity by `g(t - t_event, distance)` for some kernel `g`.

This sounds perfect for our problem. We did not use it because:

1. Hawkes models need a lot of events to fit kernels well. With only ~3,800 training events spread across 656 PSes, most PSes do not have enough history.
2. Logistic regression on PS-week features is **strictly easier to debug and explain**, and on this data size it performs comparably.
3. Operationally, the PSCA needs *PS-week probability scores*. A Hawkes model gives intensities, which need an extra conversion step.

We left Hawkes as a "try this next" item in Stage 12.

Worked example

For Mughalpura PS, our forecaster output:

- `risk_30d` = **0.94** (top of the rankings)

For a random PS in northern Punjab with no recent activity:

- `risk_30d` = **0.04**

These are the numbers the dashboard's "Next 30 Days" panel shows.

Stage 7 — Forecast total volume (Prophet)

Goal of this stage

Stages 5 and 6 give us *which PS* and *which case*. Stage 7 answers a different question: *"how many minority-related calls will the whole province see next month?"* This is useful for capacity planning — how many human reviewers / patrol units to staff.

Concept primer — what is Prophet?

Prophet is a time-series forecaster developed by Facebook (now Meta). It is a Bayesian model that decomposes a time series into three additive components plus noise:

```
y(t) = trend(t) + seasonality(t) + holidays(t) + epsilon(t)
```

Each component is modelled as a flexible function:

- **trend(t)** — a piecewise-linear function (or piecewise-logistic if you specify a cap). The bends in the line are called **changepoints**.
- **seasonality(t)** — modelled as a **Fourier series** (a sum of sines and cosines of different frequencies).
- **holidays(t)** — a binary indicator for each holiday, multiplied by a learned coefficient.
- **epsilon(t)** — Gaussian noise.

You give it a daily/weekly series of counts and it learns each component's parameters from data. It fits the model using **Stan**, a probabilistic-programming library that does Bayesian inference via MCMC or MAP.

Why it is popular

- Easy API: `m.fit(df)` then `m.predict(future)`.
- Handles missing days and outliers robustly.
- Built-in holiday support.
- Returns uncertainty intervals.

Why it can be misused

- If the trend changes mid-series (regime shift), Prophet's piecewise-linear trend struggles unless you give it changepoints manually or restrict the training window.
- The default `yearly_seasonality=True` will try to fit a yearly cycle even if you have less than two years of data — disaster.
- Default `seasonality_mode='additive'` assumes seasonal swings are the same absolute size regardless of trend level; `multiplicative` is sometimes better but also exaggerates wild swings.

We hit all three.

What we did — first attempt

Step 7.1 — Aggregate calls to monthly

```
m = (
 df.set_index("incident_date")
 .resample("M").size()
 .reset_index()
 .rename(columns={"incident_date": "ds", 0: "y"})
)
```

Now `m` is a two-column DataFrame: `ds` (month-end date) and `y` (count). Prophet requires exactly these column names.

Step 7.2 — Naive fit

```
from prophet import Prophet
p = Prophet(seasonality_mode="multiplicative")
p.fit(m)
fut = p.make_future_dataframe(periods=6, freq="M")
fcst = p.predict(fut)
```

MAPE on 2026 test months: 103.81%. Terrible.

Step 7.3 — Concept primer — MAPE

MAPE = Mean Absolute Percentage Error. For each forecast point, take the absolute percentage error from actual; average across points.

```
MAPE = 1/N * sum |y_actual - y_predicted| / |y_actual| * 100%
```

MAPE = 9.54% means on average the forecast is off by 9.54% of the actual. MAPE = 100% means the forecast is off by, on average, an amount equal to the actual — useless.

Other common error metrics:

- **MAE** (Mean Absolute Error) — same as MAPE but in raw units, not percent
- **RMSE** (Root Mean Squared Error) — penalises large errors more

Step 7.4 — Diagnose

EDA from Stage 2 told us about the **regime shift in October 2025**. The model was trying to fit a yearly seasonality across data that fundamentally changed scale halfway through. Yearly seasonality plus multiplicative noise plus a free linear trend made it explode.

Step 7.5 — Fix

Three changes:

```
post_shift = m[m["ds"] >= "2025-10-01"]
holidays = pd.DataFrame({
 "holiday": ["christmas","eid_fitr","muharram","holi","diwali"],
 "ds": ["2025-12-25","2026-03-31","2026-07-17","2026-03-04","2025-10-31"],
 "lower_window": -2, "upper_window": 2,
```

```
})
p = Prophet(
 growth="flat", # no trend
 yearly_seasonality=False, # not enough data for a yearly cycle
 weekly_seasonality=False,
 daily_seasonality=False,
 seasonality_mode="additive",
 holidays=holidays,
)
p.fit(post_shift)
```

MAPE on 2026 test months: 9.54%. Now usable.

Why we ended up with `growth="flat"`

Because in 2 months of post-shift data, you cannot reliably tell trend from noise. Prophet's default linear growth will happily extrapolate a fake upward or downward slope from a small window. Setting `growth="flat"` says "the trend is roughly constant; use the holiday regressors to explain spikes". This is **honest** modelling: tell the model what it can and cannot learn from this data.

Why we set `lower_window=-2, upper_window=2` on every holiday

Each holiday "extends" backwards and forwards by 2 days. So a holiday flag at Christmas Day = 2025-12-25 actually fires for the window [2025-12-23, 2025-12-27]. This matches our EDA finding that spikes happen "around" holidays, not only on the exact day.

Worked example

Prophet's 2026-06 forecast for Punjab-wide minority calls: **86 calls**, with a 95% confidence interval of [62, 110]. Actual count: 91. Within the CI, within 5.8% of point estimate.

Stage 8 — Wrap it all in a dashboard

Goal of this stage

A model that nobody can see is a useless model. We built **two** ways to view the predictions:

1. A **self-contained HTML dashboard** (Leaflet + Chart.js, single file) that works without any Python installed.
2. A **Streamlit web app** for interactive exploration on a laptop.

Why two?

- The HTML version is for **external presentations** — open the file in any browser, no setup. Good for stakeholders.
- The Streamlit version is for **the analyst** — interactive filtering, exploring the CSVs, easy to extend.

Tech stack — HTML dashboard

- **Leaflet** — open-source JavaScript map library
- **leaflet.heat** — heatmap layer plugin

- **leaflet.markercluster** — clustering plugin
- **Chart.js** — line + bar charts
- Vanilla JavaScript for filtering — no React, no Vue, no build step
- All assets loaded from CDN (`unpkg.com`)

The dashboard is a single HTML file with embedded JSON. Everything needed to render is in `app/dashboard.html`. Double- click it; it opens in any browser.

Concept primer — Leaflet tile layers

A web map is a grid of **tiles** — small image squares (typically 256x256 pixels) at multiple zoom levels. When you pan or zoom, the library requests the tiles it needs from a tile server.

Common tile sources:

- **OpenStreetMap (OSM)** — community-maintained, free
- **CartoDB** Positron / Dark Matter — clean grayscale, free for non-commercial use
- **Mapbox** — paid, beautiful, requires an API token
- **Google Maps** — paid, requires an API token

We used **CartoDB Positron** because it is grayscale (so our coloured heatmap stands out) and does not require a token.

```
const tiles = L.tileLayer(
 "https://{s}.basemaps.cartocdn.com/light_all/{z}/{x}/{y}{r}.png",
 { attribution: "&copy; OpenStreetMap, &copy; CartoDB" }
);
```

Concept primer — heatmap layer (kernel density)

A heatmap layer is essentially a **2D kernel density estimate**:

1. Place a small Gaussian "bump" at each event location.
2. Sum the bumps to get a density surface.
3. Colour-code the surface from low (transparent) to high (red).

`leaflet.heat` does this in a single function call:

```
const heat = L.heatLayer(points, {
 radius: 25, // pixel radius of each point's bump
 blur: 15, // how soft the edge is
 maxZoom: 17, // beyond this zoom level, switch off
 gradient: { 0.4: "blue", 0.65: "lime", 1.0: "red" },
}).addTo(map);
```

Each point is `[lat, lon, intensity]`. We weighted intensity by the RF score so high-risk cases create brighter spots than low-risk ones.

Concept primer — marker clustering

When you have 1,000+ markers on a map, drawing them all at zoom-out becomes a slow, ugly mess. **Marker clustering** groups nearby markers into a single circle showing the count, and expands them as you zoom in.

```
const cluster = L.markerClusterGroup({
 showCoverageOnHover: false,
```

```
 maxClusterRadius: 60,
});
points.forEach(p => cluster.addLayer(L.marker([p.lat, p.lon])));
map.addLayer(cluster);
```

Concept primer — Chart.js anatomy

A Chart.js chart has four key parts:

1. **type** — `line`, `bar`, `pie`, etc.
2. **data.labels** — X-axis tick labels
3. **data.datasets** — one or more series, each with a `label`, `data` array, and styling (`backgroundColor`, `borderColor`)
4. **options** — axis configuration, legend, tooltips

Example monthly-trend line chart:

```
new Chart(ctx, {
 type: "line",
 data: {
 labels: ["Jan 2024", "Feb 2024", ...],
 datasets: [{
 label: "Minority-related calls",
 data: [192, 211, 198, ...],
 borderColor: "rgb(220, 38, 38)",
 backgroundColor: "rgba(220, 38, 38, 0.15)",
 tension: 0.3,
 }]
 },
 options: { responsive: true, scales: { y: { beginAtZero: true } } }
});
```

Privacy — PII redaction in detail

The descriptions contain phone numbers, names, CNICs, sometimes addresses. For external presentation, we redact. Here is the full function with every regex explained:

```
import re

def redact_description(text: str) -> str:
 if not isinstance(text, str):
 return ""

 # 1. Pakistani mobile phone numbers: 0312-1234567, 03121234567,
 # +92-312-1234567
 text = re.sub(r"\+?9?2?[-\s]?0?3\d{2}[-\s]?\d{7}", "[PHONE]", text)

 # 2. CNICs (national ID): 12345-1234567-1
 text = re.sub(r"\b\d{5}-\d{7}-\d\b", "[CNIC]", text)

 # 3. House numbers: "House #123", "House No 45"
 text = re.sub(r"\bHouse\s*(?:#|No\.?)?\s*\d+\b", "[HOUSE_NO]", text,
 flags=re.IGNORECASE)
```

```
# 4. Street numbers: "Street 5", "St. 12"
text = re.sub(r"\b(?:Street|St\.?)\s*\d+\b", "[STREET]", text,
flags=re.IGNORECASE)

# 5. "Contact: XXXXX" blocks
text = re.sub(r"Contact\s*(?:No)?[:\.]?\s*\S+", "Contact: [REDACTED]",
text, flags=re.IGNORECASE)

# 6. SCC metadata blocks (these contain operator IDs, system codes)
text = re.sub(r"\s*SCC[\s\S]*", "", text)

# 7. "Name: First Last" patterns
text = re.sub(r"Name\s*:\s*[A-Za-z]+(?:\s+[A-Za-z]+)?",
"Name: [REDACTED]", text)

return text.strip()
```

Why every pattern in this order matters

- **Phones first** because they are the most common PII and a phone number can match looser later patterns. Catch them early.
- **CNIC pattern is very specific** (5-7-1 digits) so it can run before more general number patterns without risk of swallowing.
- **House and street patterns use `re.IGNORECASE`** because callers capitalise inconsistently.
- **SCC removed completely with `[\s\S]*`** (matches everything including newlines) because SCC blocks often contain a cascade of IDs, station codes, and timestamps that vary. Better to strip whole-block than try to redact each field.
- **Name pattern is last** because it would otherwise eat words from things like "his Name in records was" — patterns earlier in the pipeline have already pulled out structured fields.

Why "Show description" defaults to OFF

Even with full redaction, free-text descriptions sometimes leak information: "her daughter Mariam was at the corner shop" gets through because our regex does not catch first-names embedded in prose without a `Name:` prefix. The mitigation is to **default the toggle off** so descriptions only appear when an operator specifically opts in for analysis.

We considered using **spaCy NER** to catch arbitrary names but decided against it for two reasons:

1. spaCy adds 200MB+ of model files to the dependency tree.
2. NER misses Urdu / Pakistani names at a high rate; we would still have false negatives.

For external presentation the toggle-off rule is the binding control.

Tech stack — Streamlit app

- **Streamlit** — Python framework that turns a Python script into a web app
- **Pydeck** — built into Streamlit, for the map
- **Plotly** — interactive charts
- **pandas** — for filtering — drives off the CSV snapshot from Stage 1
- **One central `config.py`** so paths are easy to edit on a new machine

```python
# app/config.py
from pathlib import Path

ROOT = Path(__file__).parent.parent
DATA_DIR = ROOT / "data" / "csv_snapshot"
MODEL_DIR = ROOT / "outputs" / "models"
REPORT_DIR = ROOT / "outputs" / "reports"
```

By using `Path(__file__).parent.parent`, paths are computed relative to the location of the script. The app runs the same way whether the project lives under `/home/user/projects/` or `/tmp/test/`. No edits needed on a new laptop.

Why a "static HTML + JSON" architecture?

Alternative	Why we did not pick it
Plotly Dash	Needs a Python server running. Not portable to "open this file in your browser"
Tableau	Licence costs. Does not fit a research project
Plain matplotlib PNGs	No interactivity, no map click-through
Mapbox	Requires an API key and pings external servers (privacy concern)
Power BI	Same as Tableau

A single HTML file with bundled JSON is a **defensible default** for this kind of project. Anyone can audit the code (plain JavaScript). It runs offline. It loads in any modern browser.

Going public — the gated Streamlit deployment

Once the offline HTML dashboard is built, we want to *share* it without losing control of who can see the data. Three things have to happen for a "public, password-protected web dashboard":

1. **A login screen** — anyone with the URL gets the login page; only someone with the password sees the dashboard.
2. **Free hosting** — no server to provision or maintain.
3. **One-click redeploy** — when the data is refreshed, a single `git push` should update the live URL.

We satisfy all three with **Streamlit Community Cloud** + a tiny Streamlit wrapper around the existing HTML dashboard.

The wrapper, in 30 lines

The Streamlit script is intentionally thin. It does only three jobs:

```python
import hmac
import streamlit as st
import streamlit.components.v1 as components

def _check_login() -> bool:
 if st.session_state.get("authenticated"):
 return True
 # Show username + password fields …
```

```
    # On submit, compare against st.secrets["username"] / ["password"]
    # using hmac.compare_digest (timing-attack-safe)
    return False

if not _check_login():
    st.stop()

# Logged in - render the embedded dashboard
html = open("dashboard.html").read()
components.html(html, height=900, scrolling=True)
```

That's the whole architecture. The dashboard logic does not move to Python; we just gate access to the existing HTML.

The sanitize-and-publish pipeline

There are two flavours of `dashboard.html`:

- The **full** version (built by the dashboard builder script) lives inside the project repo. It has caller free-text descriptions, raw row- count totals, and technical model-metric captions. Stays private.
- The **public** version (produced by `scripts/sanitize.py`) lives in the deploy folder. Every `"desc":"..."` field has been replaced with a placeholder, raw-count stats are gone from the header, and technical captions are replaced with friendlier ones.

The sanitize script is two regex passes plus a few targeted substitutions:

```
PLACEHOLDER = "[redacted for public deployment]"
html = re.sub(
    r'"desc"\s*:\s*"(?:[^"\\]|\\.)*"',
    f'"desc":"{PLACEHOLDER}"',
    html,
)
# Then a list of polish substitutions: drop the "RF flagged: N" KPI,
# add the forecast-focused header, inject the forecast map layer, …
```

We chose regex over a proper HTML/JS parser because:

- The substitutions are scoped and easy to read.
- The pipeline stays idempotent — running `sanitize.py` twice produces the same output.
- The intern can edit it without learning a parser library.

Why two builds and not one

A single "always-sanitised" build would be simpler, but it loses information that is genuinely useful internally:

Audience	Build to use
PSCA case-review team (institutional access)	Full build — needs the original descriptions to triage
Project supervisor doing system audit	Full build — wants to see raw counts and metrics
Public stakeholders (donors, journalists, civic groups)	Sanitised build — gated by login
The intern handing the project forward	Sanitised build — embeds it in their portfolio

The same source code produces both. The diff is one script.

Predictions-first visual hierarchy

A late-stage redesign moved the dashboard from "*map of past incidents with stats on the side*" to "*forecast first, history as context.*" Three concrete changes:

1. **Header KPI replaced** — *" N police-stations flagged for next 30 days · Top: Mughalpura (Lahore) · score 0.94"* — the first thing the eye sees is forward-looking.
2. **Forecast map layer** — top-15 forecast PSes drawn as orange highlighted circles *above* the historical red heatmap.
3. **One-sentence subtitle** — *"Forecasts where minority-targeted incidents are most likely in the next 30 days"* — anchors the page's purpose before the user explores.

Lesson: a forecasting product is judged on whether its forecast is the visual headline. If the most prominent pixel area on screen shows past data, viewers think "this is a history dashboard" even if a forecast panel exists somewhere in a sidebar.

Verification, kept simple

Every case the dashboard shows carries two stable identifiers (`<case_id>`, `<dispatch_id>`) that exist in the source Emergency-15 database. An authorised reviewer reads either identifier from the dashboard tooltip or the triage table, looks it up in the source, and confirms what really happened. We surface these IDs prominently specifically to support this lightweight verification workflow — without ever exposing the original descriptions.

Stage 9 — Glue it all together (reproducibility)

Reproducibility checklist

When you move this project to a new machine, the order to re-create everything is:

```
# 1. Set up the environment
python -m venv venv
source venv/bin/activate # on Linux/macOS
# venv\Scripts\activate # on Windows
pip install -r requirements.txt

# 2. Re-create the source data
psql -f scripts/restore_csv_snapshot.sql # if you have DB
 # OR use CSVs directly

# 3. Re-create the per-case features
python the feature-merge script

# 4. Re-create the PS-week training table
python the project source code

# 5. Re-train the per-case classifier
python the project source code
python the project source code
```

```
# 6. Re-train the PS-week forecaster
python the project source code

# 7. Re-train Prophet
python the project source code

# 8. Predict
python the project source code
python the project source code

# 9. Build the dashboard
python the dashboard builder script

# 10. (Optional) Run the Streamlit app
streamlit run app/app.py
```

Every step writes its output as a CSV or .pkl file. **No step depends on a step it did not explicitly load from disk.** That makes debugging easy: any step can be re-run in isolation without re-running the whole pipeline.

Concept primer — virtual environments

A **virtual environment** is an isolated Python installation. It keeps your project's dependencies separate from the system Python and from other projects.

```
python -m venv venv # create
source venv/bin/activate # use
pip install pandas # installs into the venv only
deactivate # leave
```

When the user moves the project to a new laptop, they create a new venv and `pip install -r requirements.txt` re-installs the exact versions we used.

Concept primer — pinning dependencies

`requirements.txt` looks like:

```
pandas==2.2.0
scikit-learn==1.4.0
prophet==1.1.5
streamlit==1.30.0
```

The `==` pins the exact version. This is critical for reproducibility: without pinning, `pip install` on a new laptop might get a newer pandas with subtly different behaviour, and your model might score differently.

For real production we would use a lock file (`pip-compile`, `poetry`, or `uv`). For an internship project, pinning in `requirements.txt` plus a venv is enough.

A few discipline rules we followed

1. **No model trains itself on test data.** Every script splits its data by time and checks the split before training.

2. **No feature uses information from the future of its row.** This is enforced by the `compute_features_asof()` helper everywhere.
3. **Every model writes a `model_card.md`** describing what it was trained on, what its metrics are, what its limitations are.
4. **Every figure has a caption explaining what to look for in it.**
5. **No PII in the dashboard by default.** Toggle off.
6. **Every external claim in the policy brief is traceable to a metrics JSON.** No hand-waving numbers.

Stage 10 — Roadmap: future enhancements

Each item below is a known extension point that the system was designed to accommodate. Anyone continuing the project can pick any of these up next:

1. **The sentiment feature family is still a placeholder.** We designed the slot for it, but no real social-media feed is hooked up. The model's feature importance for this column is always 0 because the column is always NaN.
2. **The misinformation events list is small and hand-curated.** It should be a live feed from a media-monitoring desk.
3. **We did not try a temporal cross-validation strategy.** We split train/test by a single cut date. A more rigorous approach would be a sliding-window CV. We did not have time.
4. **The Lahore skew in the forecaster's top-50** is a real limitation. Mitigations exist (district-stratified training, or reporting top-K *per district* instead of top-50 overall) — both worth trying.
5. **No A/B test of the dashboard.** We never measured whether the forecaster's output actually changes PSCA patrol decisions. That would be the next step in a real deployment.
6. **We did not save the random seed for every component.** Most sklearn estimators have a `random_state=42` we passed. Prophet has additional `Stan` randomness that we did not lock. Reproducibility down to bit-for-bit would require seeding `Stan`.
7. **No model versioning.** We overwrite `<see project models>` each training run. A real production system would use MLflow or DVC to keep every trained model addressable.

Stage 11 — A complete worked example, top to bottom

Let us trace one row through every stage.

Step 0. Christmas Day 2025. Someone in Mughalpura, Lahore, dials Emergency-15: *"My Christian family is being threatened by neighbours after Sunday service. Contact: 0312-1234567."*

Step 1. The operator tags it `<category_lvl2> = "Religious Offences"` and writes the description. The row is inserted into `<source_call_log_table>` in the source DB. Then `build_minority_incidents.py` runs. The row passes our filter (Religious Offences tag + Christian keyword). It enters `minority_incidents` with `is_minority_targeted = 1` and `incident_date = 2025-12-25 14:30:00`.

Step 2. When we re-run EDA, the monthly bar plot now shows December 2025 elevated. The "days relative to Christmas" plot shows a spike at day 0.

Step 3. When we re-run `build_features.py`, the row gets feature columns attached:

```
ps_count_prev_30d : 7
ps_count_prev_60d : 12
ps_count_prev_90d : 21
district_count_30d : 41
days_until_christmas : 0
days_until_eid_fitr : 96
days_since_last_misinfo_district : 3
median_resp_time_ps_90d : 14.7
days_since_last_election : 687
month_sin : -0.50
month_cos : 0.87
is_Christian : 1
```

Step 4. When we re-run `predict_batch.py`, the per-case model outputs:

- `lr_score = 0.87`
- `rf_score = 0.94`

Step 5. When we re-run `psweek_dataset.py`, the row for `(PS=Mughalpura, week=2025-12-22)` updates: label = 1.

Step 6. When we re-run `predict_psweek_forward.py`, Mughalpura's forward `risk_30d` jumps from 0.78 (last week) to 0.94 (this week).

Step 7. When Prophet refits, December 2025's total goes from forecast 88 to actual 91. Within CI.

Step 8. When we re-run `dashboard.py`, the dashboard's "Next 30 Days" panel now shows Mughalpura at the top with risk 0.94. The map's red dots include this case. The KPI banner increments the "RF flagged" counter. The Christmas-Day spike is visible on the time- trend chart. The description, when toggled on, reads:

> "My Christian family is being threatened by neighbours after Sunday service. Contact: [REDACTED]"

That is the **complete loop**, from raw phone call to a coloured tile on a stakeholder's screen, with PII stripped along the way.

Stage 12 — How to extend this system (next steps)

If you want to build on what we have done, here are the next four projects worth doing, roughly in priority order:

1. Wire up a real social-media sentiment feed

Even a small one — scrape minority-keyword tweets in Punjab dialect, run a sentiment model (Twitter-XLM-RoBERTa or similar), aggregate by district-day, join into features.

The pieces:

- **Scraping**: Twitter API v2, snscrape, or Telegram-bot proxies
- **Filtering**: same keyword strategy as Stage 1 plus an Urdu word list
- **Sentiment**: `xlm-roberta-base-sentiment` via Hugging Face

- **Aggregation**: daily count of negative-sentiment posts per district
- **Join**: merge with `district` and `incident_date` in `build_features.py`

2. District-stratified PS-week forecaster

Train one model per district (or per region) instead of one Punjab- wide. Compare top-K-per-district lift to top-K-overall lift.

The Lahore skew (Stage 10 issue 4) would partially disappear because each district's model is calibrated to its own base rate.

3. Hawkes process re-test

Now that we have ~3,800 events with accurate timestamps and lat/lon, try fitting a self-exciting spatio-temporal Hawkes process and compare to the LR PS-week forecaster on top-K precision.

Libraries: `tick` (Python), or the `mhawkes` R package via rpy2.

4. Active-learning loop

Show the analyst the model's most uncertain predictions (score around 0.5), get a human label, retrain weekly. This is the single fastest way to improve model performance when labels are scarce.

The loop:

1. Score all unlabelled cases.
2. Pick the 20 closest to 0.5 (or use Bayesian uncertainty).
3. Show to a human reviewer who applies the ground-truth label.
4. Append to training set.
5. Retrain.

After 8 weeks of this, model AUC typically moves up by 5-15 points on the hardest subset of the data.

Closing thought

This system was built to a **research-grade engineering standard** — deployable, auditable, and reproducible from the data layer up. Every design choice in this document was made deliberately and is documented with its alternatives.

If you re-build this project, the most important thing to take away is not the code — the code is in the repo. It is the **way of thinking**:

- *Look at the data before you model it.*
- *Pick the simplest model that does the job.*
- *Be honest about limitations in the report.*
- *Treat privacy as a first-class design constraint, not an afterthought.*
- *Make every step reproducible from a CSV on disk.*
- *Define every term the first time you use it.*
- *Explain trade-offs, not only conclusions.*

If you take those rules into your next project, you have already beaten 80% of ML internships in the world.

Appendix A — Glossary of every technical term used

Term	Meaning
AUC	Area Under Curve — typically ROC AUC; ranking quality of a classifier
Bagging	Bootstrap aggregating; train models on random samples with replacement, average them
Bayesian inference	Statistical framework where parameters have probability distributions; prior + data give posterior
Calibration	How well predicted probabilities match observed frequencies
Changepoint	A point in time where a series' behaviour shifts
Class imbalance	When one label is much rarer than another
CNIC	Computerized National Identity Card — Pakistani national ID number
Confusion matrix	2x2 table of (actual, predicted) counts for binary classification
Cross-entropy loss	The loss function for classifiers that output probabilities
CV / cross-validation	Repeatedly split data into train/validation to estimate test performance
Decision tree	A flowchart of yes/no splits on feature values
EDA	Exploratory Data Analysis
Feature	A column of numbers fed into a model
Feature engineering	The craft of designing features
Gini impurity	A measure of how mixed the classes are in a node; range 0 to 0.5 for binary
Gradient descent	An optimisation method that steps downhill on a loss surface
Haversine	Formula for great-circle distance on a sphere
Hawkes process	A self-exciting point process; events raise the probability of future events
Imputation	Filling in missing values
Information gain	Reduction in impurity from a split
IQR	Interquartile Range (75th - 25th percentile)
L1 / L2 regularisation	Penalising large weights to prevent overfitting
Leakage (temporal)	When a feature accidentally uses future information
Logistic regression	A linear classifier that outputs probabilities via sigmoid
Logit / log-odds	log(p / (1-p)); the un-squashed score before sigmoid
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
MLE	Maximum Likelihood Estimation
One-hot encoding	Turn a category into a set of 0/1 columns
Overfitting	A model that fits training data too well and generalises poorly

Term	Meaning
Pandas	Python library for tabular data
Panel data	Many entities observed across many time periods
Pearson correlation	Linear correlation coefficient
PII	Personally Identifiable Information
PostgreSQL / Postgres	Open-source relational database
Precision	TP / (TP + FP); of predicted positives, fraction actually positive
Prophet	A time-series forecaster from Meta
Random Forest	An ensemble of decision trees with bagging and feature bagging
Recall	TP / (TP + FN); of actual positives, fraction caught
Regex	Regular expression — a pattern for matching text
Regime shift	A sudden change in time-series behaviour
Resampling	Re-aggregating a time series to a new frequency
ROC curve	TPR vs FPR as the threshold sweeps
scikit-learn (sklearn)	The standard Python ML library
Seasonality	Periodic patterns in a time series
Sigmoid	The S-shaped function 1 / (1 + exp(-x)) that maps real numbers to (0, 1)
SQL	Structured Query Language
StandardScaler	A scikit-learn transform that subtracts mean and divides by std
Stan	Probabilistic programming language used inside Prophet
Streamlit	Python framework for turning scripts into web apps
Supervised learning	Learning from labelled examples
TF-IDF	Term Frequency-Inverse Document Frequency, a text vectorisation method
Top-K precision	Of the K highest-scored predictions, fraction actually positive
Top-K lift	Top-K precision divided by base rate
Walk-forward CV	Cross-validation that respects time order
Word boundary	Regex anchor `\b` (or `\m`/`\M` in Postgres) that matches at start/end of a word

---

Authors: Internship project, Minorities Early-Warning System. Last updated: June 2026. For source code and reference materials, see the project repository under `minorities/<full project root>/`.